Consumer Onboarding Flow

From Sign-Up to First Product

How we guide new personal customers through account setup and product adoption

Consumer Onboarding Team · Q2 2026

Part 1 — Product

What the user sees, how it differs across regions, and why it matters

Eligibility · Intent · Five flows · Country rules · Drop-off

What Is the Consumer Onboarding Flow?

The bridge between **"I just signed up"** and **"I'm actively using Wise."**

What it does

- Guides customers through **personalized steps** based on who they are and where they are
- Lets them set up their multi-currency account, order a card, get bank details, or start sending money
- **Adapts dynamically** — different countries see different steps in a different order

Completed when

- Customer adopted at least one product (card, bank details, balance, send)
- OR customer explicitly skipped all options

What it is NOT

❌ Not the registration flow

Registration happens before onboarding begins

❌ Not business onboarding

Business accounts have a separate system

❌ Not a fixed screen sequence

It's highly dynamic per user

Mental model: Think of it as a state machine, not a wizard. Each call to COS re-evaluates which steps remain based on current user state — there's no stored "you are on step 4." Two users in different countries with different intents see entirely different step sequences.

When Does It Start and End?

Entry Points — How Users Enter the Flow

ENTRY POINT	BACKEND VALUE	BEHAVIOR
Registration	`REGISTRATION`	Full flow from the beginning
My Account	`ACCOUNT`	Resumes from current state
Deep link	`DEEPLINK`	Skips intent, personalizes from landing page
Content Management	`CONTENT_MANAGEMENT_WISE`	CMS-driven CTA
Receive launchpad	`LAUNCHPAD_RECEIVE`	Receive-specific flow (no card steps)
Spend launchpad	`LAUNCHPAD_SPEND`	Spend-specific flow
Young Explorer → 18	`YOUNG_EXPLORER_TURNING_18`	Transitions to full onboarding

Flow Resuming — "You Haven't Finished Setting Up"

Most users don't complete onboarding in a single session.

How resume works

1. Call `GET /v1/consumer-onboarding/flow/status`
2. Returns `IN_PROGRESS`, `COMPLETED`, or `INELIGIBLE`
3. If `IN_PROGRESS` → show resume prompt
4. Also returns `lastStep` and `shouldShowSurvey`

On resume

COS **recalculates from scratch** — doesn't jump to a saved position. Evaluates current state and returns whatever comes next.

> If eligibility changed since drop-off, the resumed flow reflects the new reality.

Per-platform behavior

	IOS	ANDROID	WEB
Prompt	My Account banner	Survey + Settings card	None
Status	`/flow/status` (7-day cache)	`/flow/status` (no cache)	`/flow` directly
Entry point	`.account`	Source mapping	URL origin

Next Best Action replaced resume banners for driving post-registration adoption. Owned by Account Discovery team — shows a personalized 3-item checklist on the Launchpad.

iOS cache gotcha: iOS caches `/flow/status` for 7 days. If a user drops off and something changes (country adds card support, eligibility shifts), iOS may show a stale resume prompt. Web has no cache — it calls `/flow` directly and either shows the flow or redirects.

Eligibility — Who Gets the Onboarding Flow?

Conditions for entering

CONDITION	WHY
Personal account	Business has separate onboarding
Not Young Explorer	Has its own experience
Country supports MCA	~80 countries eligible
Not in a Send flow	Send intent skips onboarding

~80 countries eligible

VS

~162 send-only or blocked

MCA support is the main gate — most countries lack local banking infrastructure

What happens if ineligible?

~140 countries **Send-only**

HK, AE, TR, MX, TH, NG... → Direct to transfer flow

22 countries **Blocked**

AF, CU, IR, KP, RU... → No Wise products

Indonesia **Marketing block**

Blocked from balance despite being a large market — send only

The Intent Picker — "What Do You Want to Do?"

Eligible users are asked what they want to do with Wise. This personalizes the rest of the flow.

Intent options

INTENT	WHAT HAPPENS NEXT
Send money	Transfer flow — exits onboarding
Spend with card	Card order flow
Receive / Hold	Bank details setup
Unified Onboarding	Full guided flow (most common)

When it's skipped

SCENARIO	BEHAVIOR
Brazil	Auto-routed to Unified Onboarding
Send-only countries	Auto-assigned Send
Implicit intent (e.g., card promo)	Intent pre-set
UK Current Account	CA-specific redesigned picker

Key insight: "Send money" exits onboarding entirely — the user goes straight to the transfer flow. "Unified Onboarding" is by far the most common choice and leads to Flow C (Standard). The intent picker is where the personalization divergence begins.

Send
Exits onboarding

Spend / Receive
Flow B

Unified
Flow C or D

The Five Flows

Based on eligibility and intent, users are routed to one of five distinct paths.

FLOW	NAME	WHEN	DESCRIPTION
A	Pre-Intent	Intent not yet chosen	Profile → Verify → Intent Picker → route to B/C/D/E
B	Intent-Specific	Picked Send, Spend, or Receive	Minimal steps for chosen intent only
C	Standard	Unified Onboarding, no assets	Full guided experience — **most common path**
D	Assets	Eligible for savings/investments	Like Standard but includes Assets step
E	Receive	From "Receive money" launchpad	Bank details focus — no card steps

~70%

Flow C — Standard

Most users land here via Unified Onboarding

~15%

Flow B — Intent-Specific

Send, Spend, or Receive direct

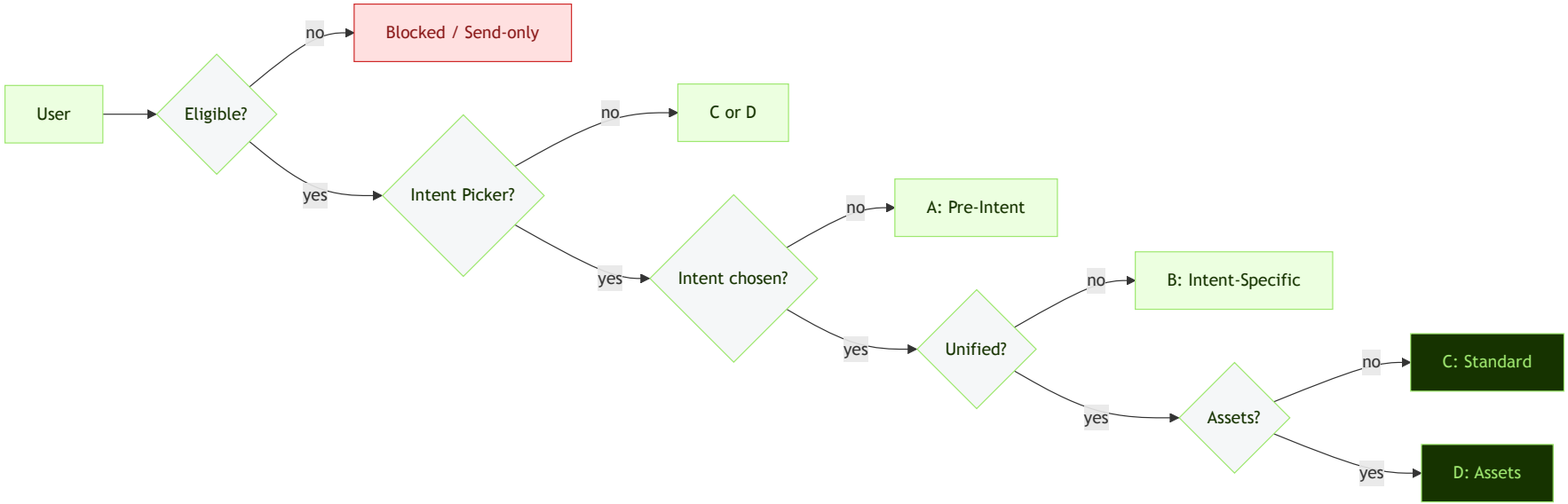
~15%

Flows A, D, E

Pre-intent, Assets, Receive launchpad

Flow Routing Decision Tree

How users get routed to one of the five flows:



Flow E (Receive) isn't shown here — it's only reachable from the Receive launchpad entry point (`LAUNCHPAD_RECEIVE`), not from the main registration path. Dark green boxes (C, D) are the most common endpoints.

The Standard Flow — Step by Step

The most common path — each step is conditionally shown.

#	STEP	WHAT THE USER SEES	SKIPPED WHEN
1	**Identity**	ID verification setup	Profile already completed
2	**Profile**	Name, DOB, address, occupation	Profile already completed
3	**Verification**	KYC / identity verification	Not required by compliance
4	**Requirements**	Legal disclosures	UK Current Account, CDD EU variant 1
5	**Currency Selection**	Pick currencies for MCA	India
6	**Discoverability**	Receive/visibility settings	India
7	**Card Intro**	Card product introduction	India, UK Current Account
8	**Card Selection**	Choose card type	Card ordered, or new card onboarding flag
9	**Balance Consent**	MCA disclosure acceptance	Non-US countries
10	**Card Order / Bank Details**	Complete card order or get bank details	Depends on card choice

Balance Consent position varies — in Assets (D) and Receive (E) flows, it appears earlier.

Step Sequences by Flow

Flow C — Standard (most common)

Identity → Profile → Verify → Requirements → **Currency Sel.** → Discover → **Card Intro** → Card Sel. → **Bal. Consent** → Card Order → Bank Details

Flow D — Assets

Identity → Profile → Verify → Requirements → **Bal. Consent** → **Assets** → Discover → **Card Intro** → Card Sel. → Card Order → Bank Details

Flow E — Receive

Identity → Profile → Verify → Reqs → **Bal. Con.** → **Assets / Curr.** → Discover → Bank Details

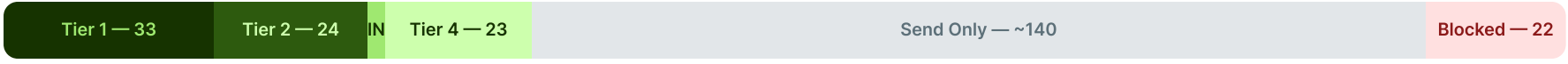
Shared — appears in all flows **Unique** — specific to this flow **Conditional** — US only / depends on eligibility

Pattern: First 4 steps are always the same (compliance block). Flows diverge after Requirements — D moves Balance Consent early, E drops card steps entirely.

Country Product Availability

TIER	PRODUCTS	#	KEY COUNTRIES	FLOW
1 — Full Suite	Send + Balance + Card + Bank Details + Assets	33	AU, BR, DE, FR, GB, US, SG	Standard or Assets
2 — No Assets	Send + Balance + Card + Bank Details	24	CA, JP, NZ, BE, MY	Standard always
3 — India	Send + Balance + Card (no Bank Details, no Assets)	1	IN	Stripped Standard
4 — No Card	Send + Balance + Bank Details	23	AR, ZA, IL, KR, SA	Standard, no card steps
5 — Send Only	Send	~140	HK, AE, TR, MX, TH, NG	Not eligible for onboarding
0 — Blocked	None	22	AF, CU, IR, KP, RU	No access

Country distribution across tiers



~81 countries get onboarding

~162 countries don't

Revenue implication: Tier 1 countries (33) generate the highest per-user revenue — full product suite means more cross-sell opportunities. India (Tier 3) has massive volume but a stripped flow. The business case for expanding Tier 4 → Tier 2 (adding card support) is active for several countries.

Country-Specific Exceptions

COUNTRY	WHAT'S DIFFERENT	WHY
US	Extra "Balance Consent" step	MCA Disclosure — legal requirement
India	Skips Currency Selection, Card Intro, Discoverability	Simplified flow — no bank details
Indonesia	Blocked from balance/card	Marketing decision — send only
Philippines	Card blocked despite product catalog	Regional override via `RegionalOnboardingFlowService`
UK	Current Account variant (permanent)	`CurrentAccountEligibilityService`: PERSONAL + GB + not Young Explorer
Brazil	Intent Picker skipped	Auto-routed to Unified Onboarding

Regional Flow Differences

REGION	BEHAVIOR
EU	Identity verification during onboarding (CDD) since Feb 2026 — ID + liveness check
UK	Current Account positioning. Home currency unboxing (personal + business from Apr 27)
Japan	eKYC — card-based "Tap to Verify" or manual entry

The Challenge — Drop-off

65%

of registered customers drop off before their first transaction

~29% verify their identity but never transact

~20% drop off at the top-up screen — **removal experiment shows +32–41% conversion lift**

15 backend steps — each is a potential drop-off point

Timeline — Key Milestones

WHEN	WHAT CHANGED
Early 2023	Unified Onboarding launches
Mid 2023	Tracking plan. Discoverability step.
Feb 2024	Assets flow integration
Mid 2024	Business onboarding added
Q1 2026	UK Current Account. Mobile deep linking
Feb 2026	EU CDD goes live
Apr 2026	Top-up removal results: V2 → +41% conversion, +7.9% LTV
Q2 2026	eKYC (Japan). Pre-registration intent

Why this matters

65% drop-off is the headline number and the reason the team exists. Every step removed or simplified directly impacts conversion.

Where we're heading

Top-up removal rollout (confirmed +41% lift). Pre-registration intent

Tracking & Metrics

Event naming

Two prefixes depending on flow position:

- `Consumer Onboarding Flow - {Step} - {Action}` — after intent picker
- `Consumer Get Started Flow - {Step} - {Action}` — before intent picker

Conventions

- **Sticky properties** — once known, attached to all subsequent events
- **Every word capitalized** in event names
- Sent to both **Mixpanel** and **Kibana** in parallel

Key properties

PROPERTY	EXAMPLE
Context	Intent Picker, Deep Link, Card Tab
Registration Country	GBR (ISO-3)
Selected Currencies	"GBP", "EGP"
Card Program	VISA_DEBIT_CONSUMER_GB_1
Did Skip Card	true / false
Result	Closed, Error, Exited

KPIs (HEART Framework)

DIMENSION	METRIC
Happiness	CES, CS contacts in 30d
Adoption	Conversion rate, first job in 30d
Task Success	Time to complete, error rate

Part 2 — Tech

How it's built, how clients talk to it, and where things live

Architecture · Kafka · Client APIs · Web / Android / iOS · Tech debt

Architecture Overview — Backend (COS)

Consumer Onboarding Service is a stateless orchestrator — a state machine that tells clients what step to show next. All service calls go through **Envoy sidecar proxies**.

Request pipeline

1. Build `OnboardingState` ← calls 15 services via Envoy
2. `OnboardingStepsFactory` ← determines flow A–E
3. `OnboardingStepDeciderService` ← filters to next step(s)
4. Return `GetOnboardingResponse` ← nextSteps + requirements + status

Key classes

CLASS	ROLE
`OnboardingState`	Eligibility flags
`StepsFactory`	Flow A–E steps
`StepDecider`	Next step filter
`StepId`	15-value enum

Step show allowance

VALUE	BEHAVIOR
ALWAYS	Re-shown every fetch
ONCE	Shown once, then done
NEVER	Conditionally hidden

Event-Driven State Updates (Kafka)

COS doesn't store step completion state — it learns about progress by calling external services and consuming **Kafka events**.

Events consumed (10 topics, key 7 shown)

TOPIC	WHAT COS LEARNS
`twcard.order.events.orderStatusChanged`	Card ordered/delivered/failed
`depositAccount.AccountCreated`	Bank details ready
`terms.consent.created`	US balance consent
`TransferService.transferStateChange`	Send flow completed
`ProfileService.profileUpdated`	Profile completed
`User.Created`	New user registered
`AccountPlans.out.PlanOrderStateChanged`	Plan order changed

Events produced

TOPIC	WHEN
`consumerOnboarding.intentPicked`	User selects intent
`Onboarding.out.BusinessOnboardingStateChanged`	Business state transition

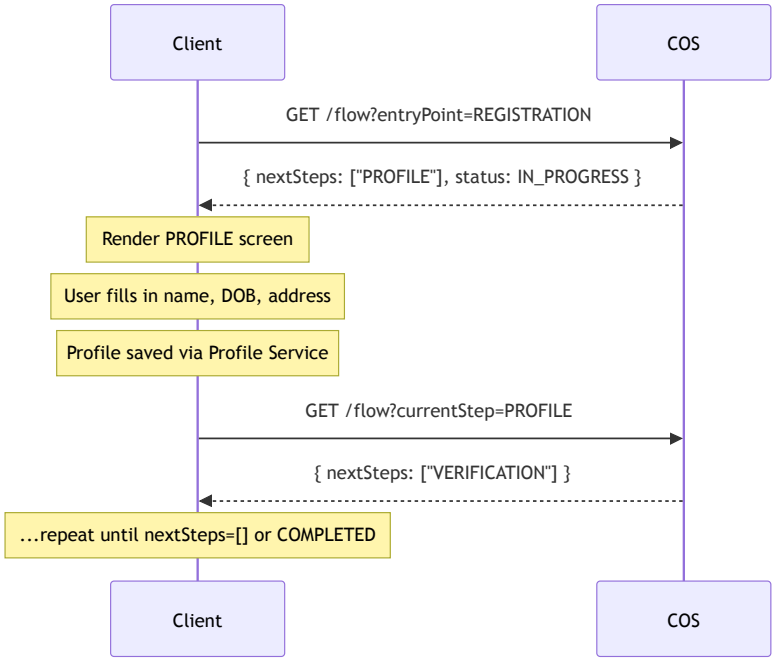
Resilience

- Kafka: `DefaultErrorHandler`, 500ms backoff, auto-commit off
- HTTP: `@Retryable` (3 attempts, 3s backoff) → fallback to "ineligible"
- Caching: **Caffeine** in-memory with configurable TTL

> **Why clients need retry:** Kafka lag — COS may not yet know a step is done.

How Clients Talk to COS

All clients follow the same polling loop. COS returns data — clients drive.



COS never stores which step the user is on — every call re-evaluates from scratch.

Client Differences — Steps & Retry

Steps per response (User-Agent based)

CLIENT	STEPS	WHY
Web	All remaining	SPA hash routing
iOS	One at a time	UIKit push/pop
Android	One at a time	Conductor fragments
Assets-eligible	One at a time	Web can't batch + assets

Web gets all steps upfront and routes via `/#hash`. Mobile clients call COS after each step completes.

Retry: client vs server

LAYER	CONFIG
Web client	`waitForNewSteps()` 5x, linear 4s→20s
Android client	`retryOn()` 5x in interactor
iOS client	`callService()` 5 consecutive
Backend (services)	`@Retryable` 3x, 3s fixed backoff
Backend (fetcher)	3x, 500ms backoff

Retry exists because of Kafka lag — client completes a step, but COS may not know yet.

The eventual consistency loop

User completes step → external service persists → Kafka event fires → COS learns about it. If the client calls COS before the event arrives, it gets the *same step back* — hence retry.

Client → Service → Kafka → COS

Web — Architecture

Repo: `consumer-onboarding-web` · **Stack:** Next.js 15, React 18, TypeScript

Flow

```
SSR (getServerSideProps)
  → First COS call
  → If COMPLETED → redirect to dashboard
  → Pass flow data as props
        ↓
Browser (CustomFlow)
  → State machine: loading → ready → redirect → complete
  → On step complete: call COS, waitForNewSteps() if same
  → Hash routing: /#profile, /#card-selection
  → Subflows: window.location.assign() (leaves SPA)
```

Shared packages

- `consumer-onboarding-api` — typed API client
- `consumer-onboarding-types` — shared TS interfaces
- `consumer-onboarding-utils` — HTTP helpers

Web-specific quirks

- Navigation: **React Router v5** `HashRouter` with `hashType="noslash"`
- State: `useRef` (non-reactive store) + `useState` (reactive) + Context
- Subflows: **Full URL redirects** — user leaves SPA, returns after
- Gets **all remaining steps** in one COS call
- Defines **18 step keys** — more than backend's 15 (includes sub-steps like `CURRENCY_SELECTION_INTRO`, `PROFILE_NAME`, `PROFILE_AGE`)

> The extra client-side step keys are a concrete example of the duplication problem.

Android — Architecture

Repo: `transferwise-android` → `feature/unified-onboarding/` · **Stack:** Kotlin, Compose, Hilt

Flow

- Entry: `UnifiedOnboardingActivity` — 11 sources (`UnifiedOnboardingSource` enum)
- Orchestrator: `UnifiedOnboardingFlowViewModel` → Conductor
- Step mapping: `FlowSteps` sealed class (14 concrete + `Unknown`)
- Subflows: `UnifiedOnboardingFlowHandoffActivity` (separate Activity)

Module structure (Clean Architecture)

```
presentation → Activity, ViewModel, UI
    ↓
core → FlowSteps sealed class, domain models
    ↓
core-impl → Retrofit, interactor (retry), repository
    ↓
android-common → Navigator, shared utilities
```

Android-specific quirks

- **Conductor** library with custom `StepAdapter` pattern, not Jetpack Navigation
- Subflows launch as **separate Activities** — onboarding pauses and resumes on return
- Unknown steps retried 5x via `retryOn()`, then fail with error
- Gets **one step at a time** from COS
- Separate `WiseUnder18Feature` + `WiseUnder18FlowActivity` for users under 18

iOS — Architecture

Repo: `transferwise-ios` → `Wise/OnboardingKit/ConsumerOnboarding/` · **Stack:** Swift, UIKit + SwiftUI

Flow

- Entry: `ConsumerOnboardingFlow` (Neptune `Flow<Result>`)
- 7 entry points: `.registration`, `.account`, `.deeplink`, `.launchpad`, `.launchpadReceive`, `.launchpadSpend`, `.contentManagement`
- Orchestrator: `StepCoordinatorImpl` with immutable `FlowState`
- Step mapping: `StepFactory` — 14 step types
- Navigation: `UINavigationController` push/pop + modal

State management

```
state.updating(\.result, .orderedCard) // returns new copy
```

Immutable updates via **keypath lenses** — functional copy-on-write.

iOS-specific quirks

- Built on **Neptune Flow Framework** — typed results, start/cancel lifecycle
- Sub-flows injected as **factory closures** (`InjectableOrderCardFlowFactory`, etc.) — coordinator doesn't know concrete types
- `FeatureService` injected via `@Dependency` but currently **unused** — all flag decisions happen server-side
- Gets **one step at a time** from COS
- Unknown steps retried up to 5 consecutive times, then `failedToFetchValidStep` error

Why iOS is the cleanest implementation: Neptune gives typed lifecycle management, keypath lenses ensure immutable state, and factory closure injection means the coordinator has zero imports from concrete flow modules. If you're looking for a reference implementation of how to integrate

Cross-Platform Comparison

RESPONSIBILITY	WEB	ANDROID	IOS
Entry point	`pages/onboarding.tsx`	`UnifiedOnboardingActivity`	`ConsumerOnboardingFlow`
Orchestrator	`CustomFlow` component	`FlowViewModel` + Conductor	`StepCoordinatorImpl`
Step registry	Object map in `constants.ts`	`FlowSteps` sealed class	`StepFactory` switch
Navigation	Hash routing (`/#step`)	Conductor (Fragments)	UINavigationController
Subflow launch	URL redirect (leaves SPA)	`HandoffActivity` (new Activity)	Factory closure → coordinator
State	`useRef` + `useState` + Context	ViewModel + Repository	Immutable state + keypath lens
Retry	`waitForNewSteps()` 5x, 4s–20s	`retryOn()` 5x	`callService()` 5 consecutive

What every client duplicates (COS returns data, not instructions)

Step registry

New step = client release on all 3 platforms

Exit conditions

Each client has its own "when to leave" — inconsistencies

Tracking

Each composes Mixpanel events independently

External Service Dependencies

COS integrates with **15 upstream services** via Envoy to compute eligibility and state.

SERVICE	WHAT IT DECIDES
Balance	MCA eligibility per country
Card Order	Card availability, orders
Card Management	Card operations
Deposit Account	Bank details currencies
Hold BFF / Assets	Investment eligibility
Mitigation Requirements	Whether verification is needed
Mitigator	Verification flow state (EU CDD)
Account Plans	Plan orders and state

SERVICE	WHAT IT DECIDES
Terms	US MCA disclosure consent
NPS	Feedback collection
Profile	Profile lookups
Route Availability	Currency routing
Bank Details Order	Bank account orders
Availability	Service availability checks
Geolocation	Country/region detection

On failure (COS is T1)

- Balance/Card down → user appears ineligible
- Hold BFF down → no assets
- Mitigation timeout → skip verification
- Terms missing → require consent (safe default)

Relevant Endpoints

Public V1 (Primary)

METHOD	PATH	PURPOSE
`GET`	`/v1/ … /flow`	Main flow — next steps + status
`GET`	`/v1/ … /flow/status`	Completion status
`GET`	`/v1/ … /eligibility`	Product eligibility
`GET`	`/v1/ … /availability`	Country availability
`GET`	`/v1/ … /intent-picker`	Intent picker data
`GET`	`/v1/ … /currencies`	Suggested currencies
`POST`	`/v1/ … /currencies`	Save currencies

Internal (S2S): `GET /users/{id}/ … /flow/status` · `PUT /users/{id}/product-adoption` · `GET /profiles/{id}/unboxing`

Public V2 / V3

METHOD	PATH	PURPOSE
`POST`	`/v2/ … /intent`	Save intent
`GET`	`/v2/ … /intent-picker`	Intent picker v2 (UK CA)
`GET`	`/v2/ … /card-types`	Card options v2
`POST`	`/v2/ … /card-types`	Select card program
`GET`	`/v3/ … /eligibility`	Eligibility (latest)

V2 Flow API — never built

Instructions-based API proposed (RFC, Jan 2024, 38 revisions) — server returns declarative instructions instead of raw state. Would eliminate client-side duplication. **Status: unimplemented.** V1 remains the only flow endpoint.

Feature Flags & Observability

Active feature flags

FLAG	PURPOSE
`CDD_EU_V2`	EU CDD verification
`account-unboxing`	Home currency unboxing
`account-structure`	Account structure (TODO: remove after full release)
`sportsbar_sponsorship_2026`	NorthAm promo — marks flow complete
`fifa_card_onboarding_*`	New card selection (3 platform flags)
`business-onboarding-free-tier-eea`	EEA business free tier

Recently removed: `current-account-onboarding-uk` — cleaned up after successful rollout. Now permanent via `CurrentAccountEligibilityService`.

Backend observability (MonitorEvent pattern)

```
Feature code → MonitorEvent → MeterMonitor
                              → Micrometer counter
                              → Structured Slf4j log
```

Key events monitored

- `ConsumerOnboardingFlowRequestEvent` — every `/flow` call
- `ConsumerOnboardingFlowStepEvent` — which step returned
- `IntentPickedMonitorEvent` — intent choices
- `CardOrderMonitorEvent` — card order lifecycle

Frontend observability

- **Mixpanel** — user behavior (all platforms)
- **Rollbar** — error capture (web)
- **Kibana** — structured logs
- **Prometheus/Grafana** — metrics + alerting

Tech Debt & Known Challenges

Business onboarding coupling

Business onboarding shares the same service since late 2024. Package-level separation (`consumer` vs `business`), but deeper layers are coupled: **shared Flyway migrations, connection pool, Spring context, and Kafka processing.**

V1 API limitations

- Clients hardcode exit logic, retry, step invocation, tracking
- **New step = mobile app release** on all platforms
- No explicit exit mechanism
- User-Agent parsing determines platform behavior

Deployment & operations

- **Continuous Deployment** via Spinnaker with automated canary analysis
- Every merge to master auto-rolls to production
- COS is **T1 (Gold maturity)**
- **Code Update Service** bot: dependency PRs every Mon/Fri

Testing reality

- **Backend:** JUnit + WireMock, parallel forks + Gradle cache
- **Web:** Cypress E2E against staging
- **Android/iOS:** Primarily **manual testing in production**
- No automated cross-platform E2E suite

Batch job

Business onboarding notification, 2x/day (`*/12h` cron) via `ServiceLeaderSelector` . Kill switch: set cron to `'-'` in config.

Highest-impact risk: Business onboarding coupling means a bad migration or Kafka consumer bug can page the consumer on-call. Mitigation: separate consumer groups are planned but not yet implemented.

Key Code Locations

Backend (`consumer-onboarding-service`)

CLASS	ROLE
`ConsumerOnboardingFlowController`	Main flow endpoint
`OnboardingStepsFactory`	Step sequence builder
`OnboardingStepDeciderService`	Next step(s) filter
`OnboardingStepId`	Step enum (15 values)
`OnboardingState`	State record
`RegionalOnboardingFlowService`	Country overrides

Web (`consumer-onboarding-web`)

- `pages/onboarding.tsx` — SSR entry
- `flows/CustomFlow/CustomFlow.tsx` — Orchestrator
- `flows/steps/constants.ts` — Step registry

Android (`transferwise-android`)

- `UnifiedOnboardingActivity` — Entry point
- `FlowSteps` — Step sealed class
- `UnifiedOnboardingFlowViewModel` — Flow VM

iOS (`transferwise-ios`)

- `ConsumerOnboardingFlow` — Main flow
- `StepCoordinatorImpl` — Step sequencing
- `StepFactory` — Step registry

New to the codebase? Start with `ConsumerOnboardingFlowController` → trace through `OnboardingStepsFactory` (picks the flow) → `OnboardingStepDeciderService` (filters steps). For client work, start at the entry points above.

Quick Reference

Links

RESOURCE	LOCATION
SOT Figma	Personal Onboarding - Source of Truth
Backend Repo	`consumer-onboarding-service`
Web Repo	`consumer-onboarding-web`
Architecture	Confluence: Architecture Overview
Tracking Plan	Confluence: Tracking Plan
Runbook	Confluence: On-call Rota and Runbook
Snowflake	`rpt_product.consumer_onboarding_flow`

Team-owned services

SERVICE	WHAT
`consumer-onboarding-service`	Backend orchestrator
`web-onboarding`	Web frontend
`availability`	Product availability API
`public-profile-service`	Public profile data
`tracking-collector`	Analytics collection

Slack

- `#consumer-onboarding-private`
- `#consumer-onboarding-public`

On-call essentials
Grafana dashboards, runbook, and alerting are linked in Confluence. COS is T1 — pages go to the on-call rota.

Analytics
Snowflake `rpt_product.consumer_onboarding_flow` is the single source of truth for conversion funnels.

Key Takeaways

1. COS is a stateless orchestrator

Every GET /flow re-evaluates from scratch. No session, no stored step. Clients drive the flow — COS returns data.

2. Five flows, one service

Users are routed to flows A–E based on eligibility, country, and intent. Flow C (Standard) is the most common path.

3. Country drives everything

Product availability, step visibility, and KYC requirements all vary by country. ~80 countries get onboarding, ~140 are send-only.

4. Three clients, one loop

Web, Android, iOS all poll COS with GET /flow. Each duplicates step registry, exit logic, retry, and tracking independently.

5. New step = mobile release

The V1 API returns raw state. Clients hardcode what to render. Adding a step means shipping on all 3 platforms.

6. Business coupling is real

Business onboarding shares the same DB, Kafka, and Spring context. A failure in one can impact the other.

Questions?

Consumer Onboarding Team · Q2 2026

`#consumer-onboarding-public`